

Actividad: Conversor y Gestión de Alumnos en Kotlin con IntelliJ IDEA

- **Módulo:** Programación Multimedia y Dispositivos Móviles
- **Duración:** 60 minutos (+15 min reto opcional)
- **Entorno de trabajo:** IntelliJ IDEA (proyecto Kotlin/JVM)

## Objetivo

Traducir la experiencia previa en Java para empezar a programar en Kotlin de forma práctica e idiomática. Trabajaremos directamente sobre una aplicación de consola Kotlin/JVM en IntelliJ IDEA para dominar la sintaxis y los conceptos clave antes de abordar las interfaces gráficas móviles.

## FASE 1: Configuración del Entorno

Para trabajar directamente con Kotlin/JVM, sin compilaciones de Android ni emuladores, crearemos un proyecto de consola en IntelliJ IDEA sino quieres instalar IntelliJ IDEA puedes utilizar Kotlin Playground donde puedes probar código Kotlin que no pida datos desde teclado. Puedes acceder en la siguiente dirección <https://play.kotlinlang.org/>

1. Abre IntelliJ IDEA.
2. Selecciona *New Project* y elige **Kotlin**. Configura el proyecto como un proyecto **Kotlin/JVM** con Gradle y un JDK instalado.
3. Crea el archivo **Main.kt** dentro de *src/main/kotlin*.
4. Escribe y ejecuta el siguiente bloque inicial para comprobar que todo funciona:

```
fun main() {  
    println("¡Entorno Kotlin configurado con éxito en IntelliJ IDEA!")  
}
```

## FASE 2: Modelado con Data Class y Null Safety

En esta fase crearemos la estructura de datos para los alumnos utilizando una `data class` y gestionaremos la posibilidad de valores nulos (?) con el operador Elvis (?:).

```
// 1. Definición de la Data Class con parámetro nulo y valor por defecto  
data class Alumno(  
    val id: Long,  
    val nombre: String,  
    var nota: Int?,  
    val email: String = "sin_email@centro.es"  
)  
  
fun main() {  
    // 2. Instanciación de objetos  
    val alumno1 = Alumno(id = 1, nombre = "Laura", nota = 8)  
    val alumno2 = Alumno(id = 2, nombre = "Carlos", nota = null)  
  
    // 3. Gestión del valor nulo con el Operador Elvis (?:)  
    val notaSegura1 = alumno1.nota ?: 0  
    val notaSegura2 = alumno2.nota ?: 0  
  
    println("${alumno1.nombre} tiene nota: $notaSegura1")  
    println("${alumno2.nombre} sin calificar, asignada por defecto: $notaSegura2")  
}
```



## FASE 3: Lógica con 'when' y Funciones

Implementaremos una función reducida de una sola línea y un evaluador de calificaciones utilizando la expresión `when` y la salida anticipada (*early return*).

```
// Función compacta de expresión única
fun esAprobado(nota: Int) = nota >= 5

// Evaluador con expresión 'when' y salida anticipada si el valor es null
fun obtenerCalificacionTexto(nota: Int?): String {
    val valorNota = nota ?: return "Sin Calificar"

    return when (valorNota) {
        in 0..4 -> "Suspendido"
        5, 6 -> "Aprobado"
        7, 8 -> "Notable"
        9, 10 -> "Sobresaliente"
        else -> "Nota No Válida"
    }
}
```



## FASE 4: Colecciones y Programación Funcional

Crearemos una lista inmutable de alumnos, iteraremos e imprimiremos el estado de cada uno y filtraremos usando `lambdas` e `it`.

```
fun main() {
    val listaAlumnos = listOf(
        Alumno(1, "Ana", 9),
        Alumno(2, "Pedro", 4),
        Alumno(3, "Elena", null),
        Alumno(4, "Lucas", 7)
    )

    println("--- INFORME DE ALUMNOS ---")

    // Recorrido e impresión formateada
    listaAlumnos.forEach { alumno ->
        val estado = obtenerCalificacionTexto(alumno.nota)
        println("ID ${alumno.id}: ${alumno.nombre} -> Estado: $estado")
    }

    // Filtrado funcional utilizando 'it'
    val aprobados = listaAlumnos.filter { it.nota != null && esAprobado(it.nota!!) }
    println("\nTotal de alumnos aprobados: ${aprobados.size}")
}
```



## RETO OPCIONAL (15 min): Estadísticas de Aula y Funciones de Extensión

Si has terminado las fases anteriores antes de tiempo, completa las siguientes ampliaciones:

- **Función de Extensión:** Crea una función llamada `esExcelente()` sobre la clase `Alumno` que devuelva `true` si la nota es igual o superior a 9.
- **Cálculo de Media:** Utiliza `mapNotNull { it.nota }` y `average()` para obtener la nota media de la clase ignorando los valores nulos.
- **Impresión:** Filtra e imprime los nombres de los alumnos excelentes.

```
// 1. Función de extensión sobre la Data Class
fun Alumno.esExcelente() = this.nota != null && this.nota!! >= 9

// 2 y 3. Añadir en el bloque main():
val notasValidas = listaAlumnos.mapNotNull { it.nota }
val notaMedia = notasValidas.average()
println("Nota media de la clase: ${"%0.2f".format(notaMedia)}")

val excelentes = listaAlumnos.filter { it.esExcelente() }
println("Alumnos excelentes: ${excelentes.map { it.nombre }}")
```

CIFP Villa de Agüimes | C/ Alcorac n.º 50, Polígono Residencial de Arinaga, Agüimes CP 35118 | Tel: 928599141 | [www.cifpvilladeaguimes.es](http://www.cifpvilladeaguimes.es)